INLAcircular: User Guide and Function Reference

Xiang Ye

Contents

1 Scope and conventions **2**

2 Installation **3**
2.1 Requirements . . . 3
2.2 Install and load . . . 3

3 Quick start **4**
3.1 Check the numerical functions . . . 4
3.2 Create example data . . . 4
3.3 Fit a circular regression . . . 4
3.4 Prepare your own circular data . . . 5

4 Circular distributions **5**
4.1 Quick comparison . . . 5
4.2 Von Mises . . . 6
4.3 Link-adjusted von Mises . . . 7
4.4 Cardioid and wrapped Cauchy densities . . . 7

5 Penalized-complexity priors **8**
5.1 Von Mises prior with uniform base: pc.vm0 . . . 8
5.2 Von Mises prior with point-mass base: pc.vminf . . . 9
5.3 Cardioid prior with uniform base: pc.card0 . . . 9
5.4 Cardioid prior with base kappa = 1/2: pc.card . . . 10
5.5 Wrapped Cauchy prior: pc.wc . . . 11
5.6 Density, probability, and vectorization rules . . . 11

6 LAvM likelihood **12**
6.1 LAvM likelihood controls . . . 12
6.2 Fitting with `inla()` . . . 13

7 Building model specifications **13**
7.1 Formula terms and their interpretation . . . 14
7.2 Mapping observations with `index()` . . . 15

8 Fitting with `inlacc()` **16**
8.1 Arguments and defaults . . . 16
8.2 Single-likelihood fit . . . 17
8.3 Joint likelihoods and copied predictors . . . 17
8.4 Returned object . . . 18

9 Modelling multivariate effects with `graphpcor` **18**
9.1 Declaring an LKJ-correlated effect . . . 19
9.2 Replicate indices . . . 20

9.3 Recovering covariance information 21

10 Results and summaries **22**
10.1 Extract predictors on the right scale 22

11 Utilities and included data **23**
11.1 Modified Bessel function 23
11.2 New York wind data 23

12 Function index **24**

13 Numerical implementation notes **25**

14 Troubleshooting and current limits **25**
14.1 Installation or function lookup 25
14.2 Model specification and numerical behaviour 25
14.3 Reproducible computation and support 26

15 Rebuilding and testing **26**
15.1 Repository layout 26
15.2 Rebuild the guide 26
15.3 Check the package 27
15.4 Identify the version used in a publication 27

16 Citation and license **28**

17 Build environment **28**



1 Scope and conventions

INLAcircular provides probability functions and a model-building interface for Bayesian circular and joint circular regression with Integrated Nested Laplace Approximation (INLA). Its core numerical routines are written in C and exposed through ordinary R functions.

This is the package's user manual and executable R vignette. The repository HTML and PDF editions are generated from the same source as the installed vignette, `vignette("INLAcircular-guide", package = "INLAcircular")`. The function reference, available through `help(package = "INLAcircular")`, documents individual arguments; this guide explains how to combine them.

Start with Installation and Quick start for a complete example. The later sections cover distribution functions, priors, structured and joint models, multivariate effects, troubleshooting, and reproducing the documentation.

The package follows these conventions:

- Angles are measured in radians.
- Location parameters are named `mu`.

- Concentration parameters are named `kappa`, including for cardioid and wrapped Cauchy distributions.
- Density functions use `log = TRUE` for log-densities.
- PC-prior distribution and quantile functions use `log.p = TRUE` for probabilities on the log scale.
- The d/p/q/r prefix has its usual R meaning: density, distribution function, quantile function, and random generation.

The core distribution and PC-prior functions can be used without `INLA`. Model fitting requires the optional `INLA` package, and LKJ multivariate random effects also require `graphpcor`.

2 Installation

2.1 Requirements

The core package requires R >= 4.1.0 and the R packages listed under `Depends` and `Imports` in `DESCRIPTION`. R installs these dependencies automatically. Additional requirements depend on the task:

Task	Additional requirements
Distribution functions and PC priors	No INLA installation
Model fitting	INLA >= 25.08.21
LKJ multivariate effects	INLA, graphpcor, and INLAtools
Build the HTML vignette	knitr, rmarkdown, and Pandoc
Rebuild both repository guides	Also pkgload and a LaTeX installation with pdflatex

Installing from source, including GitHub, compiles the C code in `src/` and requires the compiler and build tools appropriate for your R installation. Use Rtools on Windows, the R-supported command-line tools on macOS, or R development tools and a C compiler on Linux. See the official R installation manual. The bundled numerical routines do not require a separate GSL installation. The optional packages have their own R-version and operating-system requirements; the core package's minimum R version does not guarantee that every optional integration is available on that version.

The examples use CPU computation and do not require a GPU. The small simulations below are intended as installation checks; the full wind and multivariate applications can need substantially more time and memory.

2.2 Install and load

After the package is published on CRAN, install it with:

```
install.packages("INLAcircular")
```

The installation above provides the circular distributions and PC-prior functions without installing INLA. For model fitting, install optional INLA >= 25.08.21 from the current stable repository separately:

```
install.packages(
  "INLA",
  repos = c(
    getOption("repos"),
    INLA = "https://inla.r-inla-download.org/R/stable"
  )
)
```

For a development installation from GitHub:

```
install.packages("remotes")
remotes::install_github(
  "XiangYEstats/INLAcircular"
)
```

```r
library(INLAcircular)
```

Attaching INLA is optional because `INLAcircular` loads its namespace when a model-fitting function needs it. If both packages are attached and you want the compatibility `inla()` function on the search path, load `INLA` first and then `INLAcircular`.

3 Quick start

3.1 Check the numerical functions

This example runs without INLA. It checks the uniform von Mises density and a PC-prior probability/quantile round trip. Successful checks return `TRUE`.

```r
isTRUE(all.equal(dvm(0, mu = 0, kappa = 0), 1 / (2 * pi)))
#> [1] TRUE
p <- c(0.1, 0.5, 0.9)
q <- qpc.vm0(p, lambda = 2)
isTRUE(all.equal(ppc.vm0(q, lambda = 2), p, tolerance = 1e-6))
#> [1] TRUE
```

3.2 Create example data

The model examples use this simulated data frame, `dat`. It contains a circular response `x`, a continuous response `y`, an ordinary covariate `z`, and hourly node IDs from 1 to 24. The two responses share the simulated predictor `eta`; a later joint model estimates this association. The seed makes the input data reproducible.

```r
set.seed(2026)
n <- 120
hour <- rep(1:24, length.out = n)
z <- rnorm(n)
eta <- 0.5 + 0.8 * z + 0.25 * sin(2 * pi * hour / 24)
x <- rlavm(n, eta = eta, kappa = 50)
y <- 1 + 0.7 * eta + rnorm(n, sd = 0.3)
dat <- data.frame(x = x, y = y, z = z, hour = hour)
stopifnot(all(is.finite(dat$x)), all(abs(dat$x) < pi))
head(dat, 3)
#>           x        y          z hour
#> 1  1.681559 1.454250  0.5205891    1
#> 2 -0.455361 1.226221 -1.0796908    2
#> 3  1.509886 1.629874  0.1392381    3
```

Run the model examples in document order: subsequent sections reuse `dat` and the model settings. Installation commands and explicitly labelled syntax templates are not executed during rendering. By default, the CRAN vignette build executes numerical examples and data preparation, while leaving the optional model fits unevaluated. The repository renderer can execute the fits with `--run-models`, and the additional LKJ examples with `--run-lkj`; see Rebuilding and testing below. Code remains visible when a fit is not run.

This edition: the ordinary model examples were executed.

The optional LKJ examples were also executed.

3.3 Fit a circular regression

With INLA installed, specify a LAvM response block and fit it with `inlacc()`. This initial model estimates a linear effect of `z`; the structured model later in the guide also accounts for the hourly component.

```
quick_model <- likelihood(x ~ z, family = "lavm")
quick_fit <- inlacc(
  quick_model,
  data = dat,
  num.threads = "2:1",
  control.predictor = list(compute = TRUE)
)
columns <- c("mean", "sd", "0.025quant", "0.975quant")
round(quick_fit$summary.fixed[, columns], 3)
#>              mean    sd 0.025quant 0.975quant
#> Intercept_x 0.509 0.018      0.474      0.543
#> z_1         0.801 0.017      0.767      0.835
round(quick_fit$summary.hyperpar[, columns], 3)
#>                                mean    sd 0.025quant 0.975quant
#> kappa for lavm observations 7.847 0.977      6.052      9.885
```

The result contains posterior summaries for the intercept and the slope of `z`, and a natural-scale concentration `kappa`. The simulated slope is positive. Sampling variability, the omitted hourly effect, and the priors mean that estimates need not equal the generating parameters. With the inverse-tangent link, coefficients act on `eta`; they are not changes in radians per unit of covariate.

3.4 Prepare your own circular data

Supply numeric angles in radians. For LAvM fitting, use a principal interval strictly inside `(-pi, pi)`, rather than raw degrees or an unwrapped `[0, 2*pi]` representation. A common preparation is to choose a reference direction, subtract it, and wrap with `atan2(sin(x), cos(x))`. Record the reference: model coefficients describe that rotated coordinate system. Avoid placing observations exactly at the link boundary.

For example, this converts a small degree-valued sample:

```
degrees <- c(350, 10, 40)
reference <- 0
x <- degrees * pi / 180 - reference
x <- atan2(sin(x), cos(x))
stopifnot(all(is.finite(x)), all(abs(x) < pi))
x
#> [1] -0.1745329  0.1745329  0.6981317
```

Use one row per observation, numeric covariates, and the same row ordering for all responses in a joint model. Missing responses may be `NA`; the covariates and latent indices still need valid values for the rows being modelled. The examples use `x` for circular observations and `y` for an ordinary response. The transformed variable `z` in the density formula below is local mathematical notation, distinct from the example covariate.

4 Circular distributions

4.1 Quick comparison

The package supplies the full d/p/q/r interface for the von Mises and link-adjusted von Mises distributions. It supplies compiled densities for the cardioid and wrapped Cauchy distributions.

```
x <- seq(-pi, pi, length.out = 361)
densities <- cbind(
  "von Mises" = dvm(x, mu = 0, kappa = 2),
  "cardioid" = dcardioid(x, mu = 0, kappa = 0.4),
  "wrapped Cauchy" =
```

```
    dwrappedcauchy(x, mu = 0, kappa = 0.7)
)

matplot(
  x,
  densities,
  type = "l",
  col = seq_len(ncol(densities)),
  lty = seq_len(ncol(densities)),
  lwd = 2,
  xlab = "Angle (radians)",
  ylab = "Density"
)
legend(
  "topright",
  legend = colnames(densities),
  col = seq_len(ncol(densities)),
  lty = seq_len(ncol(densities)),
  lwd = 2,
  bty = "n"
)
```

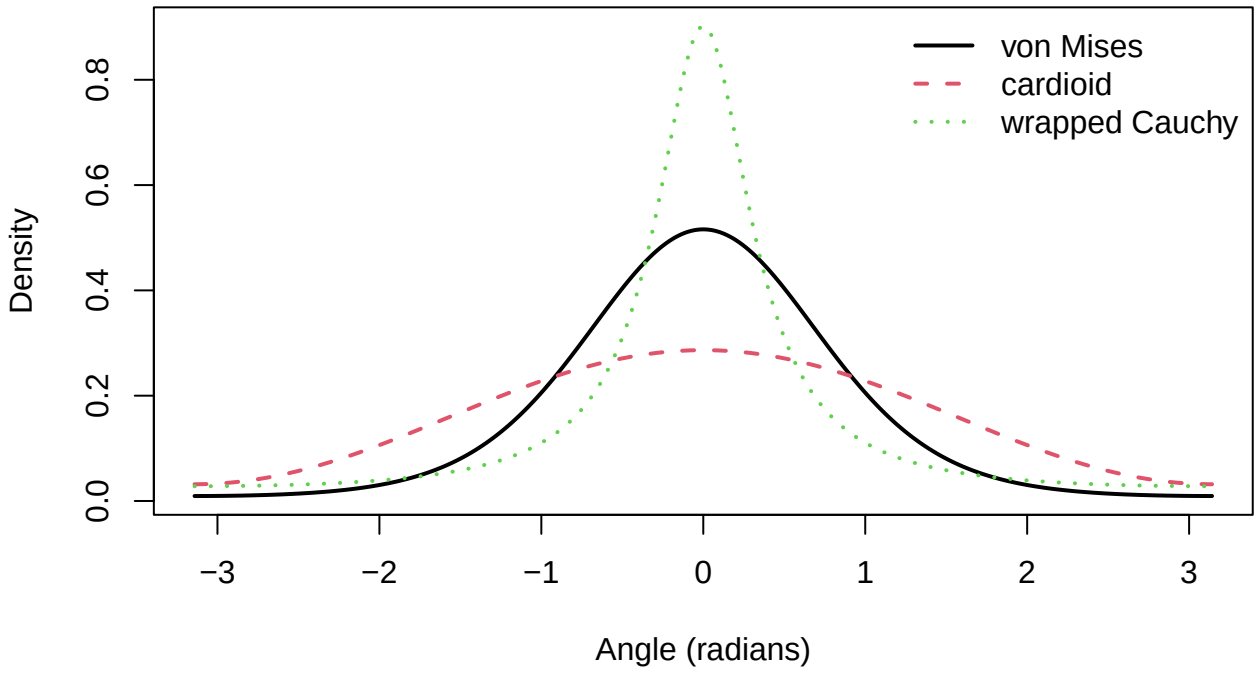


Figure 1: Three circular density families with mean direction zero.

4.2 Von Mises

`dvm()`, `pvm()`, `qvm()`, and `rvm()` evaluate the von Mises distribution with density

$$f(x \mid \mu, \kappa) = \frac{\exp\{\kappa \cos(x - \mu)\}}{2\pi I_0(\kappa)}.$$

`kappa = 0` gives the circular uniform limit. The CDF and quantile functions use a cached spline grid; `len`

controls its resolution. With `strategy = "circular"`, the CDF origin follows the mean direction. The alternative `strategy = "linear"` retains a fixed linear representation.

```
set.seed(1)
rvm(5, mu = 0, kappa = 3)
#> [1] -0.3859961 -0.2006438  0.1103994  0.8385810 -0.5174967
pvm(pi / 2, mu = 0, kappa = 2)
#> [1] 0.9625691
qvm(0.95, mu = 0, kappa = 2)
#> [1] 1.416584
```

4.3 Link-adjusted von Mises

`dlavm()`, `plavm()`, `qlavm()`, and `rlavm()` use a linear predictor `eta` and concentration `kappa`. The current direct distribution interface uses the inverse-tangent transformation

$$z = 2\arctan\{\tan(x/2) - \eta\}.$$

`lavm.link("tan")` returns the corresponding transformation and inverse functions. The `link_obj` argument in the p/q/r functions is reserved for interface compatibility; the current compiled implementation uses the inverse-tangent link.

```
set.seed(2)
rlavm(5, eta = 1.5, kappa = 3)
#> [1] 1.763773 2.058985 1.998958 1.745235 2.237412
dlavm(c(-1, 0, 1), eta = 1.5, kappa = 3)
#> [1] 0.001291978 0.003164735 0.025559191
plavm(pi / 2, eta = 1.5, kappa = 3)
#> [1] 0.07300838
qlavm(0.5, eta = 1.5, kappa = 3)
#> [1] 1.965116
```

4.4 Cardioid and wrapped Cauchy densities

The cardioid density is

$$f_{\mathrm{card}}(x \mid \mu, \kappa) = \frac{1 + 2\kappa\cos(x - \mu)}{2\pi}, \qquad 0 \le \kappa \le \frac{1}{2}.$$

Use `dcardioid()`, or its short alias `dcard()`.

The wrapped Cauchy density is

$$f_{\mathrm{wc}}(x \mid \mu, \kappa) = \frac{1 - \kappa^2}{2\pi\{1 + \kappa^2 - 2\kappa\cos(x - \mu)\}}, \qquad 0 \le \kappa < 1.$$

Use `dwrappedcauchy()`, or its short alias `dwc()`. The endpoint `kappa = 1` is a point-mass limit: the compiled density reports `Inf` at the mean direction and zero elsewhere. Finite-parameter likelihoods should use `kappa < 1`.

Both functions recycle `x`, `mu`, and `kappa` to a common length.

```
x <- c(-pi / 2, 0, pi / 2)
dcardioid(x, mu = 0, kappa = c(0, 0.25, 0.5))
#> [1] 0.1591549 0.2387324 0.1591549
```

```
dwrappedcauchy(x, mu = 0, kappa = 0.7, log = TRUE)
#> [1] -2.909998 -0.103276 -2.909998
```

5 Penalized-complexity priors

A PC prior assigns an exponential distribution with rate `lambda` to a distance from a simpler base model. Larger `lambda` puts more prior mass near the base. All five prior families expose matching density, CDF, quantile, and random-generation functions.

Help topic	Base model	Support for kappa	Special behavior
pc.vm0	von Mises uniform limit, kappa = 0	[0, Inf)	Continuous
pc.vminf	von Mises point-mass limit	[0, Inf)	Atom exp(-lambda) at zero
pc.card0	cardioid uniform model, kappa = 0	[0, 1/2]	Normalized truncated exponential distance
pc.card	cardioid base, kappa = 1/2	[0, 1/2]	Atom at zero; integrable singularity at 1/2
pc.wc	wrapped Cauchy uniform model, kappa = 0	[0, 1)	Continuous; open upper endpoint

For a monotone distance $d(\kappa)$, an untruncated continuous PC prior has density

$$\pi(\kappa) = \lambda \exp\{-\lambda d(\kappa)\} \left| d'(\kappa) \right|.$$

When the attainable distance range is bounded, the remaining exponential tail must be handled explicitly. `pc.card0` truncates and normalizes the exponential distance law; `pc.vminf` and `pc.card` collapse the remaining tail to a boundary atom. The density functions below return densities with respect to Lebesgue measure. A boundary atom is therefore represented by the CDF and RNG, not added to the value returned by a density function.

5.1 Von Mises prior with uniform base: pc.vm0

Write

$$A(\kappa) = \frac{I_1(\kappa)}{I_0(\kappa)}, \qquad A'(\kappa) = 1 - A(\kappa)^2 - \frac{A(\kappa)}{\kappa},$$

where the continuous limit is $A'(0) = 1/2$. The distance from the circular uniform model is

$$d_0(\kappa) = \sqrt{\kappa A(\kappa) - \log I_0(\kappa)}.$$

For $\kappa > 0$, the density and CDF are

$$\boxed{\pi_0(\kappa) = \lambda e^{-\lambda d_0(\kappa)} \frac{\kappa A'(\kappa)}{2 d_0(\kappa)}}, \qquad F_0(\kappa) = 1 - e^{-\lambda d_0(\kappa)}.$$

The density has the finite endpoint limit $\pi_0(0) = \lambda/2$. This is a continuous distribution with no atom at zero. Use `dpc.vm0()`, `ppc.vm0()`, `qpc.vm0()`, and `rpc.vm0()`.

```r
probability <- c(0.1, 0.5, 0.9)
q0 <- qpc.vm0(probability, lambda = 2)
cbind(
  probability = probability,
  quantile = q0,
  recovered = ppc.vm0(q0, lambda = 2)
)
#>      probability  quantile recovered
#> [1,]         0.1 0.1054703       0.1
#> [2,]         0.5 0.7266853       0.5
#> [3,]         0.9 6.6791614       0.9

dpc.vm0(c(0, 0.5, 2), lambda = 2)
#> [1] 1.00000000 0.57267102 0.09577906
```

5.2 Von Mises prior with point-mass base: pc.vminf

The distance from the point-mass limit $\kappa \to \infty$ is

$$d_\infty(\kappa) = \sqrt{1 - A(\kappa)}.$$

The continuous density for $\kappa > 0$ and the full CDF are

$$\boxed{\pi_\infty^{\text{cont}}(\kappa) = \lambda e^{-\lambda d_\infty(\kappa)} \frac{A'(\kappa)}{2 d_\infty(\kappa)}}, \qquad F_\infty(\kappa) = e^{-\lambda d_\infty(\kappa)}.$$

Because $d_\infty(0) = 1$, the full law has boundary mass $\Pr(\kappa = 0) = e^{-\lambda}$. `dpc.vminf()` returns only the continuous component, whose right-hand endpoint limit is $\lambda e^{-\lambda}/4$. `ppc.vminf()`, `qpc.vminf()`, and `rpc.vminf()` include the atom.

```r
lambda <- 2
atom_vm <- exp(-lambda)
c(computed = ppc.vminf(0, lambda), expected = atom_vm)
#>  computed  expected
#> 0.1353353 0.1353353
dpc.vminf(c(0, 0.5, 2), lambda)
#> [1] 0.06766764 0.09193625 0.09948602
qpc.vminf(c(atom_vm / 2, (1 + atom_vm) / 2), lambda)
#> [1] 0.000000 6.527716
```

5.3 Cardioid prior with uniform base: pc.card0

Let

$$s(\kappa) = \sqrt{1 - 4\kappa^2}, \qquad d_0(\kappa) = \sqrt{1 - s(\kappa) + \log\{(1 + s(\kappa))/2\}}.$$

The distance is bounded above by $D_0 = \sqrt{1 - \log 2}$. Therefore pc.card0 uses the normalized, truncated exponential density

$$\boxed{\pi_0(\kappa) = \frac{\lambda e^{-\lambda d_0(\kappa)}}{1 - e^{-\lambda D_0}} \frac{2\kappa}{\{1 + s(\kappa)\} d_0(\kappa)}}.$$

Its CDF is

$$F_0(\kappa) = \frac{1 - \exp\{-\lambda d_0(\kappa)\}}{1 - \exp(-\lambda D_0)}.$$

Use `dpc.card0()`, `ppc.card0()`, `qpc.card0()`, and `rpc.card0()`. The density at zero is evaluated by continuity and equals $\lambda/\{1 - e^{-\lambda D_0}\}$. At $\kappa = 1/2$, it equals $\lambda e^{-\lambda D_0}/[D_0\{1 - e^{-\lambda D_0}\}]$.

```
probability <- c(0.1, 0.5, 0.9)
quantile <- qpc.card0(probability, lambda = 2)
cbind(
  probability = probability,
  quantile = quantile,
  recovered = ppc.card0(quantile, lambda = 2)
)
#>      probability   quantile recovered
#> [1,]         0.1 0.03465088       0.1
#> [2,]         0.5 0.20172637       0.5
#> [3,]         0.9 0.43368208       0.9
dpc.card0(c(0, 0.25, 0.5), lambda = 2)
#> [1] 2.986217 1.892776 1.780359
```

5.4 Cardioid prior with base kappa = 1/2: pc.card

The distance to the concentrated cardioid base is

$$d_{1/2}(\kappa) = \sqrt{1 - 2\kappa - s(\kappa) + \log\{1 + s(\kappa)\}}.$$

It decreases from $\sqrt{\log 2}$ at zero to zero at the base. Its continuous density is

$$\boxed{\pi_{1/2}^{\text{cont}}(\kappa) = \lambda e^{-\lambda d_{1/2}(\kappa)} \frac{1 + s(\kappa) - 2\kappa}{\{1 + s(\kappa)\} d_{1/2}(\kappa)}}.$$

The full CDF is

$$F_{1/2}(\kappa) = \exp\{-\lambda d_{1/2}(\kappa)\}.$$

Consequently, the distribution has boundary mass $\exp\{-\lambda\sqrt{\log 2}\}$ at `kappa = 0`. `dpc.card()` returns the continuous density component. `ppc.card()`, `qpc.card()`, and `rpc.card()` include the atom. The continuous density tends to `Inf` at `kappa = 0.5`, but the singularity is integrable. At zero, its continuous component is $\lambda e^{-\lambda\sqrt{\log 2}}/\sqrt{\log 2}$; this value is distinct from the atom probability.

```
lambda <- 2
atom_card <- exp(-lambda * sqrt(log(2)))
c(computed = ppc.card(0, lambda), expected = atom_card)
#> computed expected
#>  0.18917  0.18917
qpc.card(c(atom_card / 2, (1 + atom_card) / 2), lambda)
#> [1] 0.0000000 0.4024443
dpc.card(c(0, 0.25, 0.49), lambda)
#> [1] 0.4544326 1.0445693 6.6278466
```

5.5 Wrapped Cauchy prior: pc.wc

For the wrapped Cauchy family,

$$d_{\mathrm{wc}}(\kappa) = \sqrt{-\log(1-\kappa^2)}$$

maps [0, 1) to the full nonnegative distance axis. Its density is

$$\boxed{\pi_{\mathrm{wc}}(\kappa) = \lambda e^{-\lambda d_{\mathrm{wc}}(\kappa)} \frac{\kappa}{(1-\kappa^2) d_{\mathrm{wc}}(\kappa)}}.$$

The CDF is

$$F_{\mathrm{wc}}(\kappa) = 1 - \exp\{-\lambda d_{\mathrm{wc}}(\kappa)\}.$$

Use `dpc.wc()`, `ppc.wc()`, `qpc.wc()`, and `rpc.wc()`. The density limit at zero is `lambda`; the density diverges as `kappa` approaches one. Interior-probability quantiles (`0 < p < 1`) and RNG output remain strictly below one. There is no atom at one; the value `dpc.wc(1, lambda)` is the limiting density convention.

```
q_wc <- qpc.wc(c(0.1, 0.5, 0.9), lambda = 2)
cbind(quantile = q_wc, recovered = ppc.wc(q_wc, lambda = 2))
#>        quantile recovered
#> [1,] 0.05264373       0.1
#> [2,] 0.33642236       0.5
#> [3,] 0.85692657       0.9

set.seed(3)
rpc.wc(5, lambda = 2)
#> [1] 0.09179213 0.70196661 0.23947530 0.19660997 0.43736859
dpc.wc(c(0, 0.5, 0.9), lambda = 2)
#> [1] 2.0000000 0.8503661 0.5585013
```

5.6 Density, probability, and vectorization rules

Every family follows the same call pattern:

```
dpc.NAME(kappa, lambda, log = FALSE)
ppc.NAME(q, lambda, log.p = FALSE)
qpc.NAME(p, lambda, log.p = FALSE)
rpc.NAME(n, lambda)
```

The PC-prior functions recycle concentrations or probabilities against `lambda`. The rate must always contain positive, finite values. With `log.p = TRUE`, CDFs return log-probabilities and quantiles accept log-probabilities. The density flag is `log`, and both flags must be one non-missing logical value. There is no `lower.tail` argument; all probabilities are lower-tail.

Values and `lambda` recycle to their common maximum length, with a warning for non-multiple lengths. For random generation, a vector `n` requests `length(n)` draws; otherwise its scalar value is converted to an integer. The generators use R's RNG stream. Apart from the documented limiting-endpoint conventions, densities are zero outside their mathematical support, while invalid probabilities produce `NaN`.

Names such as `pc.card0` are help topics and prior labels, not callable R functions. Use the corresponding d/p/q/r functions.

```r
dpc.card0(c(0.1, 0.2, 0.3), lambda = c(1, 2, 3), log = TRUE)
#> [1] 0.7622848 0.7219704 0.4663980
qpc.wc(log(c(0.1, 0.5, 0.9)), lambda = 2, log.p = TRUE)
#> [1] 0.05264373 0.33642236 0.85692657
```

6 LAvM likelihood

Use `family = "lavm"` with `inla()` or `inlacc()` from INLAcircular. The LAvM likelihood is implemented using INLA's compiled `cloglike` interface. The package handles response preparation automatically.

The following settings can be supplied as `control.family` in `inla()` or as `family.setting` in `likelihood()`:

```r
lavm_setting <- list(
  link = "inverse.tangent",
  hyper = list(
    kappa = list(
      initial = log(50),
      prior = "pc.vminf",
      param = c(0.5, 0.5),
      fixed = FALSE
    )
  )
)
```

6.1 LAvM likelihood controls

The LAvM settings accept the following controls:

- `link` is `"inverse.tangent"` (the default), `"scaled.logit"`, or `"scaled.probit"`.
- `hyper$kappa$initial` is one finite initial value for $\theta = \log(\kappa)$. The default is 6; it is not a natural-scale concentration.
- `hyper$kappa$prior` is `"pc.vminf"` (the default) or `"pc.vm0"`.
- `hyper$kappa$param = c(u, alpha)` contains two values strictly between zero and one. Here u is a threshold for the mean resultant length $A(\kappa) = I_1(\kappa)/I_0(\kappa)$, and the calibration is $\Pr\{A(\kappa) > u\} = \alpha$.
- `hyper$kappa$fixed = TRUE` fixes $\log(\kappa)$ at `initial`; the default is `FALSE`.

If κ_u solves $A(\kappa_u) = u$, `pc.vm0` uses

$$\lambda = -\frac{\log(\alpha)}{d_0(\kappa_u)},$$

whereas `pc.vminf` uses

$$\lambda = -\frac{\log(1-\alpha)}{\sqrt{1-u}}.$$

This `c(u, alpha)` interface differs from the standalone d/p/q/r functions, which take `lambda` directly.

For `pc.vminf`, this calibration comes from the full PC law, including its atom at `kappa = 0`. A fitted INLA hyperparameter is finite $\theta = \log(\kappa)$, so model fitting uses only the continuous component transformed to the θ scale, including its Jacobian. It has no separate discrete state at $\theta = -\infty$. Treat `c(u, alpha)` as the rate calibration for the fitted continuous prior, not as a claim that the fitted hyperparameter retains the boundary atom.

6.2 Fitting with `inla()`

A single LAvM model can be fitted with `inla()` without using `inlacc()`. Loading INLAcircular makes its `inla()` function available. If you also attach INLA, load it first and INLAcircular second. The example below uses `x` for the circular response and `z` for an ordinary covariate, both taken from the simulated `dat`.

```
library(INLAcircular)

fit_short <- inla(
  x ~ z,
  family = "lavm",
  data = dat,
  num.threads = "2:1",
  control.family = lavm_setting
)

columns <- c("mean", "sd", "0.025quant", "0.975quant")
round(fit_short$summary.fixed[, columns], 3)
#>               mean    sd 0.025quant 0.975quant
#> (Intercept) 0.509 0.018      0.474      0.543
#> z           0.801 0.017      0.767      0.835
round(fit_short$summary.hyperpar[, columns], 3)
#>                               mean    sd 0.025quant 0.975quant
#> kappa for lavm observations 7.847 0.977      6.052      9.885
```

The package reports the concentration on the natural `kappa` scale. Use `INLAcircular::inla()` if you need to select this function explicitly. Calls with other likelihood families are forwarded to INLA.

7 Building model specifications

`inlacc()` uses a formula-building layer to assemble one or more likelihood blocks into a stacked INLA model. Create one response block with

```
likelihood(formula, family, family.setting = NULL)
```

Its arguments are:

- `formula`: a two-sided formula for one response. The left side should be a single column name in `data`.
- `family`: one INLA likelihood name. Use `"lavm"` for the package's link-adjusted von Mises likelihood; other names are passed to INLA.
- `family.setting`: the `control.family` list for this block. For LAvM it has the structure documented in the preceding section.

A single likelihood object can be passed directly to `inlacc()`. A joint model is a list. Likelihood order determines response-block order and, where relevant, multivariate component order.

```
x_model <- likelihood(
  x ~
    intercept(name = "alpha0", mean = 0, sd = 2) +
    covariate(z, name = "alpha_z", mean = 0, sd = 1) +
    f(
      hour_effect,
      model = "rw1",
      cyclic = TRUE,
      constr = TRUE,
      scale.model = TRUE,
      hyper = list(
```

```
        prec = list(
          prior = "pc.prec",
          param = c(1, 0.01)
        )
      )
    ),
  family = "lavm",
  family.setting = lavm_setting
)
```

7.1 Formula terms and their interpretation

The right side may contain ordinary numeric fixed-effect expressions and the following markers:

Term	Meaning inside `inlacc()`
`1`	Automatic block-specific intercept named `Intercept_<response>`
`0` or `-1`	Remove the automatic intercept
`intercept(...)`	Named intercept with an optional Gaussian prior
ordinary term such as `z` or `log(z)`	Block-specific fixed effect evaluated in `data`
`covariate(...)`	Fixed effect or automatically/explicitly copied response predictor
`f(...)`	Standard INLA latent-effect term, passed into the assembled formula

Use numeric columns and expressions for ordinary terms. The assembler does not reproduce every `model.matrix()` feature for factors, offsets, or special response expressions. Use unique custom coefficient names across likelihoods unless a shared fixed coefficient is intentional.

7.1.1 `intercept()`

```
intercept(name = NULL, mean = NULL, sd = NULL)
```

- `name` changes the coefficient label; its default is `Intercept_<response>`.
- `mean` and `sd` set the Gaussian prior. A missing value inherits the corresponding global fixed-effect setting.
- A formula `0` or `-1` removes the automatic intercept, while an explicit `intercept()` adds the custom one.

This is a formula marker. It returns `NULL` when called by itself because `inlacc()` interprets its unevaluated call.

7.1.2 `covariate()`

```
covariate(
  var,
  name = NULL,
  mean = NULL,
  sd = NULL,
  initial = NULL,
  fixed = FALSE,
  predictor = NULL
)
```

- `var` is an unquoted variable or expression evaluated in `data`. It is an argument, not a separate function named `var()`.
- `name` supplies a stable coefficient and display name.
- `mean` and `sd` independently override the Gaussian prior for an ordinary fixed effect.
- `predictor = NULL`, the default, copies the latent predictor when `var` is an untransformed response in another likelihood block; otherwise it creates an ordinary observed fixed effect.
- `predictor = TRUE` explicitly requests copying. It is an error if `var` is not an untransformed response from another block.
- `predictor = FALSE` always creates an ordinary fixed effect from the observed values, including when `var` is another response.
- `initial` and `fixed` control the INLA copy-scaling coefficient. They apply to copied predictors and are not used for ordinary fixed effects.

For a copied predictor, provide both `mean` and `sd` to customize its scaling prior. If either is absent, the current implementation uses its default $N(0, 31.62^2)$ scaling prior. Any response used directly as a bare term on another block's right side is also copied automatically. To use that response's observed values instead, write `covariate(response, predictor = FALSE)`. A transformed expression such as `log(response)` is an ordinary observed fixed effect under `predictor = NULL` or `FALSE`; requesting `predictor = TRUE` for a transformed expression is an error because this interface can copy only a named source predictor.

7.1.3 `f()` and standard INLA options

INLAcircular does not export a separate `f()`. Write a bare `f(...)` in an `inlacc()` formula. Its syntax is the R-INLA latent-effect syntax, including `model`, `hyper`, `constr`, `scale.model`, `cyclic`, `group`, `control.group`, `replicate`, `values`, and `copy`. Use `INLA::inla.doc()` for details of the selected latent model.

The term is preserved in the assembled INLA formula. INLAcircular additionally does the following:

- supplies an index column from `data` or `index()`;
- creates `1:nrow(data)` when an index is otherwise absent;
- injects or replaces `values` for the matching first index when a complete `process.id` is given;
- records common model and hyperprior information for summaries; and
- rewrites the special `model = "iidkd_LKJ"` form described later.

Because the preprocessor recognizes the call name literally, use `f(...)`, not `INLA::f(...)`, in these formulas.

7.2 Mapping observations with `index()`

```
index(var, data.id, process.id = NULL)
```

Here `data.id` and `process.id` are arguments of `index()`, not separate functions.

- `var` is a character string matching the first argument of an `f()` term.
- `data.id` maps each observation row to a latent-process node and should normally have `nrow(data)` entries.
- `process.id` optionally gives the complete ordered node set. When supplied, it becomes the INLA `values` argument for an `f()` term whose first argument matches `var`, replacing any existing `values` argument. When it is `NULL`, INLA infers represented values from the observation index. A mapping can supply `data.id` for a first, group, or replicate variable, but `process.id` applies only to a matching first index.

```
hour_index <- index(
  var = "hour_effect",
  data.id = c(1, 2, 2, 4),
  process.id = 1:24
)
```

```
hour_index
#> $var
#> [1] "hour_effect"
#>
#> $data.id
#> [1] 1 2 2 4
#>
#> $process.id
#>  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
#> [24] 24
```

Pass one object directly or several in a list. This example supplies the hourly mapping for `x_model`, using node IDs from 1 to 24:

```
latent_indices <- list(
  index("hour_effect", data.id = dat$hour, process.id = 1:24)
)
```

Add one `index()` object for each additional latent process. Each mapping must match an index actually used in the model.

If a mapped name is not used by any `f()` first, group, or replicate argument, it is ignored with a warning. For an ordinary shared effect, `index(var, data.id = "likelihood")` sets its index to the likelihood-block number. A later section documents the different special LKJ form `process.id = "likelihood"`.

8 Fitting with `inlacc()`

The complete call is

```
inlacc(
  model,
  data,
  latent.index = NULL,
  LKJ.eta = 5,
  control.family = NULL,
  control.fixed = NULL,
  control.inla = list(cmin = 0, compute.initial.values = TRUE),
  metrics = FALSE,
  verbose = FALSE,
  ...
)
```

8.1 Arguments and defaults

- `model` is one `likelihood()` object or a non-empty list of them.
- `data` is a data frame containing every simple response column, fixed-effect variable, and ordinary latent index. All likelihood blocks use the same number and ordering of observation rows; missing responses are represented with `NA` and handled by INLA.
- `latent.index` is one `index()` object or a list of them.
- `LKJ.eta` is one positive finite LKJ shape shared by every `iidkd_LKJ` group. It defaults to 5 and is irrelevant when no such group is present.
- `control.family` optionally overrides block-level `family.setting`. Supply one nested control list per likelihood, even for a single block. When no override exists for a block, that block's `family.setting` is used.

- `control.fixed` is an INLA fixed-effect control list. The package defaults are mean 0 and precision 0.001, equivalent to SD about 31.62. Its `mean` and `prec` may be scalars or INLA named lists with a `default` entry; existing named entries are preserved. Priors specified in `intercept()` or `covariate()` override the matching coefficients.
- `control.inla` is passed to `INLA::inla()`. By default, `cmin` is 0 and `compute.initial.values` is `TRUE`. Supplying another list replaces that whole default, so include either setting explicitly when it remains required.
- `metrics = TRUE` requests CPO, DIC, WAIC, and INLA configuration output. The default is `FALSE`.
- `verbose` is passed directly to `INLA::inla()`.
- `...` accepts further INLA arguments. Common examples are `control.predictor`, `control.compute`, `control.mode`, and `num.threads`. A supplied `control.compute` is merged field by field with the values implied by `metrics`.

Do not pass `formula` or `family` through `...`: those names would replace the internally assembled values. Use the formal `control.family` and `control.fixed` arguments only for their documented controls; the stacked formula and data are owned by `inlacc()`.

8.2 Single-likelihood fit

```
fit <- inlacc(
  model = x_model,
  data = dat,
  num.threads = "2:1",
  latent.index = index(
    "hour_effect",
    data.id = dat$hour,
    process.id = 1:24
  ),
  control.predictor = list(compute = TRUE),
  metrics = TRUE
)
```

8.3 Joint likelihoods and copied predictors

This example fits both simulated responses in `dat`. The first block is `x_model`, including its hourly effect. The second uses the first block's latent predictor through an INLA copy term. Because `x` is the response of another likelihood block, `covariate(x)` selects this behavior automatically.

```
joint_model <- list(
  x_model,
  likelihood(
    y ~
      intercept(name = "beta0", mean = 0, sd = 5) +
      covariate(
        x,
        name = "beta_x",
        mean = 0,
        sd = 1,
        initial = 0,
        fixed = FALSE
      ),
    family = "gaussian",
    family.setting = list(
      hyper = list(
        prec = list(prior = "pc.prec", param = c(1, 0.01))
```

```
      )
    )
  )
)

joint_fit <- inlacc(
  model = joint_model,
  data = dat,
  latent.index = latent_indices,
  num.threads = "2:1",
  control.predictor = list(compute = TRUE),
  metrics = TRUE
)
joint_summary <- summary(joint_fit)
round(joint_summary$fixed_par[, c("mean", "SD", "2.5%", "97.5%")], 3)
#>          mean    SD  2.5% 97.5%
#> alpha0  0.508 0.006 0.496 0.520
#> alpha_z 0.794 0.006 0.782 0.806
#> beta0   1.033 0.028 0.977 1.088
round(joint_summary$random_par[
  "beta_x", c("mean", "SD", "2.5%", "97.5%"), drop = FALSE
], 3)
#>         mean   SD  2.5% 97.5%
#> beta_x 0.697 0.03 0.638 0.757
```

The Gaussian block contains the copied predictor without an additional effect of `z`: duplicating the same linear information can make the copy coefficient difficult to identify. Here `beta_x` describes the association with the circular block's latent `eta`, not with its observed angle in radians. Copy coefficients are INLA hyperparameters, so the package summary places `beta_x` in `random_par` rather than `fixed_par`.

Writing `predictor = TRUE` in that term is equivalent but makes the copy request explicit. To regress on the observed angles instead of their latent predictor, use `covariate(x, predictor = FALSE)`.

Set `control.predictor = list(compute = TRUE)` when fitted linear predictors are needed. The custom summary warns when predictor computation is particularly relevant, including models with copied covariates.

8.4 Returned object

The result inherits from both `inlacc` and INLA result classes, so ordinary INLA components remain available. The package also records:

- the assembled formula, stacked data, effective families, and family controls in `inlacc_meta`;
- fixed-effect, hyperparameter, prior, and latent-process mappings used by `summary.inlacc()`;
- the names of copied predictors; and
- `lkj_modes` and `inlacc_meta$LKJ`, both always lists and empty when no LKJ effect is present.

LAvM concentration marginals are normally transformed from $\log(\kappa)$ to κ and renamed in the fitted object. Raw INLA fields such as `summary.fixed`, `summary.hyperpar`, `marginals.fixed`, `marginals.hyperpar`, `summary.fitted.values`, and `cpu.used` remain available.

9 Modelling multivariate effects with `graphpcor`

INLAcircular integrates one specific graphpcor model: a dense LKJ-correlated IID vector shared across likelihood blocks. It does not expose graphpcor's general graph-structured or tree-structured model interfaces.

Install the optional dependencies before using this feature:

```
install.packages(c("graphpcor", "INLAtools"))
```

The feature inherits graphpcor's own R-version and dependency requirements. Both packages are checked at run time.

Some INLAtools versions first look for a separate graphpcor shared library and warn if it is absent. An INLA executable with a built-in LKJ model can still complete the fit. Use `verbose = TRUE` to confirm which model was selected; otherwise a compatible compiled graphpcor library is needed. A path warning alone is not evidence of a successful or failed fit. The examples below proceed to inspect the fitted covariance parameters.

9.1 Declaring an LKJ-correlated effect

Use the exact special syntax

```
f(
  i,
  model = "iidkd_LKJ",
  pc.prior.u = 1,
  pc.prior.alpha = 0.5
)
```

in at least two likelihood blocks. Terms with the same bare first argument (`i` above) form one multivariate process. Component order follows the likelihood objects supplied to `inlacc()`.

- A process may occur at most once in each likelihood.
- Its dimension is the number of likelihoods containing the term and must be at least two.
- The same index name cannot also be used by another `f()` model.
- `LKJ.eta` is supplied once to `inlacc()`, not as `eta` inside `f()`.
- `prior.param` is not supported for this special term.

The per-component scale prior is passed to graphpcor as `sigma.prior.reference = pc.prior.u` and `sigma.prior.probability = pc.prior.alpha`. The defaults are 1 and 0.5. The reference must be positive; the probability may be in $[0, 1]$ or `NA`. The PC-scale calibration used by the implementation is $\Pr(\sigma > u) = \alpha$, equivalently $\lambda = -\log(\alpha)/u$. Values `0`, `1`, and `NA` fix that component's standard deviation at `u` rather than estimating a scale parameter.

This separate simulation creates two responses with correlated latent intercepts, using one replicate per row. It does not overwrite `dat`.

```
set.seed(2027)
n_lkj <- 60
b <- matrix(rnorm(2 * n_lkj), ncol = 2) %*%
  chol(matrix(c(0.25, 0.15, 0.15, 0.36), nrow = 2))
dat_lkj <- data.frame(
  x = rlavm(n_lkj, eta = b[, 1], kappa = 50),
  y = b[, 2] + rnorm(n_lkj, sd = 0.2)
)
```

```
multivariate_model <- list(
  likelihood(
    x ~ intercept(name = "alpha0") +
      f(
        i,
        model = "iidkd_LKJ",
        pc.prior.u = 1,
        pc.prior.alpha = 0.5
      ),
```

```
    family = "lavm",
    family.setting = lavm_setting
  ),
  likelihood(
    y ~ intercept(name = "beta0") +
      f(
        i,
        model = "iidkd_LKJ",
        pc.prior.u = 2,
        pc.prior.alpha = 0.5
      ),
    family = "gaussian"
  )
)

fit_lkj <- inlacc(
  multivariate_model,
  data = dat_lkj,
  num.threads = "2:1",
  LKJ.eta = 5,
  control.predictor = list(compute = TRUE)
)
#> Warning in normalizePath(shlib):
#> path[1]="/home/xiang/Rlibrary/graphpcor/libs//graphpcor.so": No such file
#> or directory
```

`LKJcc()` performs the preprocessing internally: it collects the terms, constructs graphpcor's cgeneric model, rewrites the formulas, and creates component and replicate indices. Most users should not call it directly. Its return value contains the rewritten formulas, generated models, block data, process metadata, and unconsumed ordinary index mappings.

9.2 Replicate indices

By default, observations receive replicate IDs `seq_len(nrow(data))`; no `i` column is required. To use explicit replicate IDs, supply the special mapping

```
replicate_id <- seq_len(nrow(dat_lkj))
lkj_index <- index(
  var = "i",
  data.id = replicate_id,
  process.id = "likelihood"
)

fit_lkj <- inlacc(
  multivariate_model,
  data = dat_lkj,
  latent.index = lkj_index,
  num.threads = "2:1",
  control.predictor = list(compute = TRUE),
  LKJ.eta = 5
)
#> Warning in normalizePath(shlib):
#> path[1]="/home/xiang/Rlibrary/graphpcor/libs//graphpcor.so": No such file
#> or directory
```

In this special form, `data.id` must contain one positive integer per observation and becomes the INLA replicate index. The component index is generated from likelihood order. An explicit `replicate = some_name` must use a bare variable name. It may be written in one or more special `f()` terms, but every explicit name within a group must agree. That name controls the generated replicate column; its values still come from this mapping or the default row sequence.

9.3 Recovering covariance information

`fit_lkj$lkj_modes[["i"]]` stores internal posterior-mode values, not a full posterior correlation summary. When every scale is free (`0 < pc.prior.alpha < 1`), the first p entries for a p-dimensional group are log standard deviations and the remaining $p(p-1)/2$ entries parameterize the correlation matrix. The example above has this all-free form.

```
p <- 2L
theta_i <- fit_lkj$lkj_modes[["i"]]

sd_mode <- exp(theta_i[seq_len(p)])
correlation_mode <- graphpcor::basecor(
  theta_i[-seq_len(p)],
  p = p
)$base

sd_mode
#> Theta1 for i Theta2 for i
#>    0.5273132    0.6587518
correlation_mode
#>           [,1]      [,2]
#> [1,] 1.0000000 0.4313327
#> [2,] 0.4313327 1.0000000
```

For a mixture of free and fixed scales, reconstruct the complete SD vector before extracting the correlation parameters. The following code also works for the all-free example above:

```
process_i <- fit_lkj$inlacc_meta$LKJ[["i"]]
u <- process_i$pc.prior.u
alpha <- process_i$pc.prior.alpha
fixed_scale <- is.na(alpha) | alpha <= 0 | alpha >= 1
n_free <- sum(!fixed_scale)

theta_i <- fit_lkj$lkj_modes[["i"]]
log_sd_mode <- log(u)
log_sd_mode[!fixed_scale] <- theta_i[seq_len(n_free)]

n_cor <- p * (p - 1L) / 2L
cor_index <- n_free + seq_len(n_cor)
correlation_mode <- graphpcor::basecor(
  theta_i[cor_index],
  p = p
)$base

sd_mode <- exp(log_sd_mode)
```

For multiple LKJ groups, use the corresponding first-argument name as the list key. Component order is always the likelihood order used to build the group. These are point estimates at the internal posterior mode; use INLA's marginal or sampling tools when full uncertainty summaries are required.

10 Results and summaries

`summary()` dispatches to `summary.inlacc()` for fitted models:

```
summary(object, decimal = 3, ...)
```

It returns an `inlacc_summary` object with the following components:

- `fixed_par`: fixed-effect posterior summaries and displayed Gaussian priors;
- `likelihood_par`: transformed likelihood parameters, including natural-scale LAvM concentration and Gaussian standard deviations;
- `random_par`: latent-process hyperparameters and their prior labels;
- `time.summary`: setup, computation, post-processing, and total times;
- `metrics`: available marginal likelihood, DIC, and WAIC summaries; and
- `needs_pred_warning`: whether the fitted model would benefit from `control.predictor = list(compute = TRUE)`.

The `decimal` value controls printing, not stored posterior precision. `print.inlacc_summary(x, decimal = NULL, metrics = TRUE)` can override the number of displayed decimal places and suppress the metrics section. Timing components use `print.inlacc_time(x)`. These methods are normally called through `summary()` and `print()` rather than directly.

```
fit_summary <- summary(fit, decimal = 3)
round(fit_summary$fixed_par[, c("mean", "SD", "2.5%", "97.5%")], 3)
#>          mean    SD  2.5% 97.5%
#> alpha0  0.508 0.006 0.496 0.520
#> alpha_z 0.794 0.006 0.781 0.806
```

The selected columns above give a compact table. For the full console printout, including priors and timing, use:

```
print(fit_summary, metrics = TRUE)
```

Because an `inlacc` result remains an INLA result, the original posterior summaries, marginals, fitted values, predictor values, model criteria, and timing fields remain accessible with standard INLA component names. CPO values remain on the raw fit even though the custom printed summary does not create a separate CPO table.

10.1 Extract predictors on the right scale

For a single response, the first `nrow(dat)` predictor rows correspond to the observations. In a joint fit, requested response blocks are stacked in likelihood order. Copied predictors can introduce additional auxiliary rows after those response blocks, so do not assume every returned row is a new observation.

```
n <- nrow(dat)
eta_x <- joint_fit$summary.linear.predictor[seq_len(n), ]
eta_y <- joint_fit$summary.linear.predictor[n + seq_len(n), ]
head(eta_x[, c("mean", "sd", "0.025quant", "0.975quant")], 3)
#>                     mean         sd 0.025quant 0.975quant
#> Predictor.001  1.0013750 0.02445055  0.9533796  1.0494444
#> Predictor.002 -0.2169084 0.02507968 -0.2660918 -0.1675586
#> Predictor.003  0.7936973 0.02436255  0.7458085  0.8415297
head(eta_y[, c("mean", "sd")], 3)
#>                     mean         sd
#> Predictor.121 1.7310025 0.03418197
#> Predictor.122 0.8814635 0.03538824
#> Predictor.123 1.5861953 0.03166700
```

For the inverse-tangent LAvM link, `2 * atan(eta)` is the conditional location/median parameter. Transforming posterior quantiles of `eta` gives quantiles of this parameter. It is not the same calculation as a posterior predictive circular mean or a prediction interval for a new angle.

```
location <- 2 * atan(
  as.matrix(eta_x[, c("0.025quant", "0.5quant", "0.975quant")])
)
head(location, 3)
#>               0.025quant   0.5quant 0.975quant
#> Predictor.001  1.5230723  1.5721572  1.6190385
#> Predictor.002 -0.5201312 -0.4272547 -0.3320327
#> Predictor.003  1.2816263  1.3417843  1.3991120
```

When data were rotated before fitting, add the recorded reference direction and wrap again to report locations in the original coordinates. Inspect `summary.linear.predictor` for `eta`; do not assume `summary.fitted.values` contains circular means.

The fitted LAvM interface does not currently provide a CDF for INLA's PIT diagnostics. Requesting `metrics = TRUE` does not make LAvM PIT available. Check available CPO values and their failure indicators, and interpret DIC or WAIC comparisons only for models fitted to the same responses and data.

11 Utilities and included data

11.1 Modified Bessel function

`bessel_i(x, nu, expon.scaled = FALSE)` evaluates the modified Bessel function of the first kind using C routines adapted from GSL. R is only the interface. The order `nu` must be finite and nonnegative; fractional orders are supported for nonnegative `x`. Negative `x` is supported for integer orders, with the usual parity rule; otherwise it produces `NaN` and a warning. Set `expon.scaled = TRUE` to return $I_\nu(x)\exp(-|x|)$, which avoids overflow for large x. Very large orders can reduce the accuracy of unscaled values near the transition between exponential growth and decay; such inputs produce an accuracy warning when the estimated loss is significant.

```
bessel_i(c(0, 1, 5), nu = 0)
#> [1]  1.000000  1.266066 27.239872
bessel_i(1, nu = 0.5)
#> [1] 0.9376749
bessel_i(1000, nu = 0, expon.scaled = TRUE)
#> [1] 0.01261724
```

11.2 New York wind data

`wind_newyork` contains 8,759 hourly climate-normal records for JFK International Airport (station USW00094789), supplied with the New York wind application. Its UTC calendar uses 2010 as a reference year; these are not 8,759 observed weather measurements taken in 2010. The first record is January 1 at 01:00, so the reference calendar lacks January 1 at 00:00.

The principal fields are `HLY.WIND.VCTDIR` (circular direction in radians), `HLY.WIND.VCTSPD` (wind speed), `HLY.TEMP.NORMAL` (temperature), and `hour` (0 through 23). `DATE`, `datetime`, `month`, and `day` complete the eight columns. See `?wind_newyork` for the stored schema. The included metadata does not specify the original speed/temperature units or the climate-normal baseline period; establish those from the upstream dataset before reporting physical-unit effects.

```
data("wind_newyork", package = "INLAcircular")
dim(wind_newyork)
#> [1] 8759    8
```

```r
utils::head(names(wind_newyork), 4)
#> [1] "DATE"            "HLY.WIND.VCTDIR" "HLY.WIND.VCTSPD" "HLY.TEMP.NORMAL"
```

This preparation uses January's records, centres direction about its sample circular mean, standardizes the temperature covariate, and creates the hourly node IDs expected by `x_model`:

```r
wind <- wind_newyork[wind_newyork$month == 1, ]
x <- as.numeric(wind$HLY.WIND.VCTDIR)
reference <- atan2(mean(sin(x)), mean(cos(x)))
x <- atan2(sin(x - reference), cos(x - reference))
wind_dat <- data.frame(
  x = x,
  z = as.numeric(scale(wind$HLY.TEMP.NORMAL)),
  hour = wind$hour + 1L
)
stopifnot(
  all(is.finite(as.matrix(wind_dat))),
  all(abs(wind_dat$x) < pi),
  all(wind_dat$hour %in% 1:24)
)
head(wind_dat, 3)
#>            x         z hour
#> 1 0.06664463 0.2422834    2
#> 2 0.06664463 0.1668680    3
#> 3 0.11900451 0.0537449    4
```

```r
wind_fit <- inlacc(
  x_model,
  data = wind_dat,
  latent.index = index(
    "hour_effect", data.id = wind_dat$hour, process.id = 1:24
  ),
  num.threads = "2:1",
  control.predictor = list(compute = TRUE)
)
round(wind_fit$summary.fixed[, columns], 3)
#>          mean    sd 0.025quant 0.975quant
#> alpha0  0.000 0.001     -0.001      0.001
#> alpha_z 0.011 0.002      0.008      0.014
round(wind_fit$summary.hyperpar[, columns], 3)
#>                                    mean      sd 0.025quant 0.975quant
#> kappa for lavm observations  692.214  36.298    623.341    765.865
#> Precision for hour_effect   1013.198 316.709    516.695   1750.319
```

This is a small demonstration of the interface, not a complete analysis of weather variability. The repository's `tests/manual/06-wind-new-york.R` contains the larger joint circular-Gamma application. That developer script requires INLA and `circular` and may take longer to run.

12 Function index

Area	Functions
Von Mises distribution	dvm, pvm, qvm, rvm
Link-adjusted von Mises	lavm.link, dlavm, plavm, qlavm, rlavm

Area	Functions
Other circular densities	dcardioid (dcard), dwrappedcauchy (dwc)
Von Mises PC prior, uniform base	dpc.vm0, ppc.vm0, qpc.vm0, rpc.vm0
Von Mises PC prior, point-mass base	dpc.vminf, ppc.vminf, qpc.vminf, rpc.vminf
Cardioid PC prior, uniform base	dpc.card0, ppc.card0, qpc.card0, rpc.card0
Cardioid PC prior, kappa = 1/2 base	dpc.card, ppc.card, qpc.card, rpc.card
Wrapped Cauchy PC prior	dpc.wc, ppc.wc, qpc.wc, rpc.wc
Model specification	likelihood, intercept, covariate, index
Model fitting	inlacc, inla
Multivariate preprocessing	LKJcc
Results	summary.inlacc, print.inlacc_summary, print.inlacc_time
Numerical utility	bessel_i
Included data	wind_newyork

Function-level help is available with `?function_name`. For names containing dots, quote the topic if needed, for example `help("dpc.card0")`.

13 Numerical implementation notes

The cardioid and wrapped Cauchy routines use compiled C throughout their numerical path. Endpoint limits, log-density calculations, inverse transforms, and random generation are not implemented as R formulas. Cancellation-prone expressions use algebraic reductions, `log1p()`, `expm1()`, and local series near difficult boundaries.

14 Troubleshooting and current limits

14.1 Installation or function lookup

- If a fit reports that INLA is unavailable, install it from the stable repository shown above and check `packageVersion("INLA")`. Loading INLAcircular alone is enough for its fitting functions to find INLA.
- If `inla()` selects the wrong function after loading other packages, call `INLAcircular::inla()` explicitly. Keep `family = "lavm"` for this interface.
- If the installed vignette is missing after a GitHub installation, rebuild with `build_vignettes = TRUE` or read the repository HTML/PDF. Binary installations do not require the documentation tools to read a built guide.
- A compiler error during source installation concerns the R build environment. Check the platform-specific tools before trying model fits. Missing Pandoc or LaTeX affects documentation building, not the numerical API.

14.2 Model specification and numerical behaviour

- Validate radians, the chosen reference direction, and the principal angle interval before fitting. Numerical clamping at a link boundary is not a replacement for wrapping circular data correctly.
- Supply LAvM concentration settings under `hyper$kappa`. Its `initial` value is `log(kappa)`. The `c(u, alpha)` prior calibration is distinct from the `lambda` argument of the standalone PC-prior functions.
- Ensure each latent index has one entry per data row and that its values belong to the specified `process.id` set. The wind data's raw hour field needs `+ 1L` when the latent nodes are `1:24`.
- Use numeric fixed-effect columns or explicitly prepared indicator columns. The model assembler does not implement every factor, offset, or special response-expression feature of R's `model.matrix()`.

- If a model has poorly identified effects, inspect duplicated predictors, copied terms, and prior sensitivity. Start with a simpler model, scale continuous covariates, and use `verbose = TRUE` to inspect INLA diagnostics.
- LKJ effects require `graphpcor` and `INLAtools`, at least two likelihood blocks, and matching index names. The exposed interface is a dense correlated IID effect; it is not the full graphpcor model catalogue.

INLA supplies approximate Bayesian inference. This guide demonstrates API usage, not a guarantee of inferential accuracy for every data-generating process. Check modelling assumptions and sensitivity for the application. The distribution p/q/r functions currently use the inverse-tangent LAvM transformation, even though fitting supports additional links. The fitted positive concentration prior has the boundary limitation described in the LAvM controls section, and LAvM PIT is currently unavailable.

14.3 Reproducible computation and support

The fitting examples request `num.threads = "2:1"`. BLAS thread count is an INLA option, not an argument to `inlacc()` or `inla()`:

```
INLA::inla.setOption(blas.num.threads = 1L)
```

Numerical summaries and run times may differ with R, INLA, dependency versions, platform, and thread settings. Preserve seeds, input preparation, priors, and the environment information at the end of this guide.

Report reproducible problems at the issue tracker. Include a small data example or simulation, the complete model call, the error text, and `sessionInfo()`. For maintainer contact, use `maintainer("INLAcircular")`. Do not include private data in a public issue.

15 Rebuilding and testing

15.1 Repository layout

The repository follows the standard R package structure:

Location	Purpose
`R/`	Public R functions and model assembly
`src/`	Compiled numerical routines and fitting integration
`man/`	Generated function help and CRAN reference-manual source
`vignettes/`	This guide's executable R Markdown source
`tools/render-guide.R`	Rebuild the repository HTML and PDF guides
`tests/`	Automated numerical and interface regression tests
`tests/manual/`	Longer developer models and application checks
`data/`, `inst/`	Packaged data, citations, and licensing notices
`output/html/`, `output/pdf/`	Rendered repository documentation

The numerical layer is callable without INLA. The modelling layer translates `likelihood()`, formula terms, and index mappings into an INLA model; `inlacc()` returns the INLA result with package-specific summaries and metadata. LKJ preprocessing is delegated to `graphpcor` when requested.

15.2 Rebuild the guide

From a clone or downloaded source tree, install the documentation packages:

```
install.packages(c("knitr", "rmarkdown", "pkgload"))
```

Install Pandoc and a LaTeX distribution containing `pdflatex` for both formats. Then run one of these commands from the repository root:

```
Rscript tools/render-guide.R
Rscript tools/render-guide.R --run-models
Rscript tools/render-guide.R --run-lkj
```

The first command builds the lightweight edition. The second also executes the ordinary INLA examples, including the small wind fit. The third includes the LKJ examples and requires all optional modelling dependencies. These commands use the current source package, fail on missing requested dependencies, and overwrite the two rendered guide files. Model execution occurs separately for HTML and PDF. The output records which examples ran; successful completion means all enabled chunks evaluated without error.

The RStudio Knit button and VS Code knitting shortcut use the same YAML render function. To build only the installable HTML vignette through the standard package workflow, use `R CMD build` as described next; that path does not require LaTeX for this HTML vignette.

15.3 Check the package

Install the declared dependencies before running a package check. INLA is installed from its separate repository; `graphpcor`, `INLAtools`, and the documentation packages are optional dependencies for the complete checks. In a terminal at the repository root:

```
R CMD build .
R CMD check --as-cran INLAcircular_1.0.0.tar.gz
```

Use the archive name printed by `R CMD build` if the package version differs. The standard check runs function examples, automated tests, and vignette checks. PDF reference-manual checking also requires LaTeX. To run a specific longer developer example after installing this source version:

```
R CMD INSTALL .
Rscript tests/manual/01-direct-inla-lavm.R
Rscript tests/manual/02-joint-circular-linear.R
```

See `tests/manual/README.md` in the repository for the remaining scripts and their dependency requirements. These longer applications are deliberately outside the automatic CRAN test path. The CI configuration declares checks for Linux, Windows, and macOS, plus a minimum-R core-dependency job. Consult the actual run logs before claiming that a particular version passed on all platforms.

Automated tests cover numerical boundaries, missing values, probability and quantile consistency, optional-INLA behaviour, LAvM controls, copied predictors, and LKJ preprocessing. Passing these checks demonstrates those tested behaviours; it is not a benchmark or proof of model accuracy for every application.

15.4 Identify the version used in a publication

Repository links under `main` change over time. For a paper or archived analysis, record the release tag or full commit SHA, the package version, the dependency versions, and the guide built from that same source. A local Git checkout can report its commit with:

```
git rev-parse HEAD
```

Use the corresponding commit or release URL for the source and guide links in the article's software metadata. Record any local modifications as well; the version in `DESCRIPTION` alone does not identify an uncommitted source tree. A versioned archive DOI, when one has been created, can supplement the repository link. This guide does not assert that an archive or release tag already exists.

16 Citation and license

Use the package's citation information for the circular-regression framework and the PC-prior methods:

```
citation("INLAcircular")
toBibtex(citation("INLAcircular"))
```

The method references are Ye, Van Niekerk, and Rue's circular regression framework and PC priors for circular models. The installed `CITATION` file is the maintained source for their bibliographic entries. Cite the software version used as well as the applicable methods.

INLAcircular is distributed under GNU GPL version 3 or later, as recorded in `DESCRIPTION`. The full license and third-party notices are available through the repository and in an installed package:

```
basename(system.file("LICENSES", "GPL-3", package = "INLAcircular"))
#> [1] "GPL-3"
basename(system.file("COPYRIGHTS", package = "INLAcircular"))
#> [1] "COPYRIGHTS"
```

17 Build environment

These values describe the environment that rendered this edition, not a claim that the listed versions are the only supported versions. An optional package being installed does not mean its examples were executed; see the evaluation statement in Quick start.

```
packages <- c("INLAcircular", "INLA", "graphpcor", "INLAtools")
versions <- vapply(packages, function(package) {
  if (requireNamespace(package, quietly = TRUE)) {
    as.character(packageVersion(package))
  } else {
    "not installed"
  }
}, character(1))
data.frame(package = packages, version = unname(versions))
#>        package version
#> 1 INLAcircular   1.0.0
#> 2         INLA 26.11.8
#> 3    graphpcor  0.1.25
#> 4    INLAtools   0.1.4
```

```
sessionInfo()
#> R version 4.6.2 (2026-10-29)
#> Platform: x86_64-redhat-linux-gnu
#> Running under: Fedora Linux 44 (Workstation Edition)
#>
#> Matrix products: default
#> BLAS/LAPACK: FlexiBLAS OPENBLAS-OPENMP;  LAPACK version 3.12.1
#>
#> locale:
#>  [1] LC_CTYPE=C.UTF-8       LC_NUMERIC=C           LC_TIME=C.UTF-8
#>  [4] LC_COLLATE=C.UTF-8     LC_MONETARY=C.UTF-8    LC_MESSAGES=C.UTF-8
#>  [7] LC_PAPER=C.UTF-8       LC_NAME=C              LC_ADDRESS=C
#> [10] LC_TELEPHONE=C         LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C
#>
#> time zone: UTC
#> tzcode source: system (glibc)
```

```
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] INLAcircular_1.0.0
#>
#> loaded via a namespace (and not attached):
#>  [1] sass_0.4.10         generics_0.1.4      class_7.3-24
#>  [4] fmesher_0.8.0.9004  KernSmooth_2.23-27  lattice_0.23-1
#>  [7] digest_0.6.39       magrittr_2.0.5      evaluate_1.0.5
#> [10] grid_4.6.2          INLAtools_0.1.4     pkgload_1.5.2
#> [13] fastmap_1.2.0       rprojroot_2.1.1     jsonlite_2.0.0
#> [16] Matrix_1.7-5        pkgbuild_1.4.8      e1071_1.7-17
#> [19] DBI_1.3.0           tinytex_0.59        numDeriv_2016.8-1.1
#> [22] jquerylib_0.1.4     cli_3.6.6           rlang_1.3.0
#> [25] units_1.0-1         graphpcor_0.1.25    INLA_26.11.08
#> [28] splines_4.6.2       withr_3.0.3         cachem_1.1.0
#> [31] yaml_2.3.12         otel_0.2.0          tools_4.6.2
#> [34] parallel_4.6.2      MatrixModels_0.5-4  dplyr_1.2.1
#> [37] Deriv_4.2.0         vctrs_0.7.3         R6_2.6.1
#> [40] proxy_0.4-29        lifecycle_1.0.5     classInt_0.4-11
#> [43] pkgconfig_2.0.3     desc_1.4.3          pillar_1.11.1
#> [46] bslib_0.10.0        glue_1.8.1          Rcpp_1.1.2
#> [49] sf_1.1-3            xfun_0.57           tibble_3.3.1
#> [52] tidyselect_1.2.1    knitr_1.51          htmltools_0.5.9
#> [55] igraph_2.3.3        rmarkdown_2.31      compiler_4.6.2
```